

Fenix FEM

Plataforma de Análisis por Elementos Finitos en Python

Sigla: FF-MU

Manual de Usuario

Sintaxis del archivo YAML, catálogos de componentes,
ejemplos completos y workflow de ejecución

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

13 de mayo de 2026

Índice general

Sobre este manual	III
1. Introducción y Configuración del Entorno	1
1.1. Filosofía del Proyecto	1
1.1.1. Modelo Data-Driven	1
1.2. Requisitos del Sistema	1
1.2.1. Dependencias	1
1.3. Herramientas Externas Recomendadas	2
1.4. Ejecución de un Análisis	2
2. Arquitectura del Programa	3
2.1. Separación Elemento $\leftrightarrow$ Material	3
2.2. Auto-Registro de Componentes	3
2.3. Validación Temprana del Input	4
3. Archivo de Entrada YAML	5
3.1. Geometría: <code>nodes</code> y <code>elements</code>	5
3.2. Importación de Malla Gmsh: <code>mesh</code>	6
3.2.1. Mapeo Avanzado: <code>mesh_physical_groups</code>	6
3.3. Materiales	6
3.4. Condiciones de Frontera	7
3.4.1. Por nodo: <code>boundary_conditions_by_node</code>	7
3.4.2. Por coordenada: <code>boundary_conditions_by_coord</code>	7
3.4.3. Por grupo físico: <code>boundary_conditions_by_group</code>	7
3.4.4. Restricciones multipunto (MPC): <code>linear_constraints</code>	7
3.5. Cargas Puntuales	8
3.6. Fuerzas de Cuerpo	8
3.6.1. Forma general: <code>body_force</code>	8
3.6.2. Caso particular — peso propio: <code>gravity + density</code>	8
3.6.3. Detalles transversales	9
3.7. Solver	9
3.7.1. Bloque <code>convergence</code>	9
3.8. Salida: <code>output</code>	11
4. Catálogo de Elementos Finitos	12
4.1. Elementos 1D	12
4.1.1. Armaduras: <code>Truss2D</code> / <code>Truss2DCorot</code> / <code>Truss3D</code> / <code>Truss3DCorot</code>	12
4.1.2. Cables: <code>Cable2DCorot</code> / <code>Cable3DCorot</code>	13

4.1.3.	Marcos 2D: <code>Frame2DEuler</code> / <code>Frame2DTimoshenko</code> / <code>Frame2DEulerCorot</code> . . .	13
4.1.4.	Marco 3D: <code>Frame3D</code>	14
4.2.	Elementos 2D	15
4.2.1.	Cuadrilátero Bilineal: <code>Quad4</code>	15
4.2.2.	Triángulo de Deformación Constante: <code>Tri3</code>	15
4.2.3.	Cuadrilátero Serendípito de Orden 2: <code>Quad8</code>	15
4.2.4.	Cuadrilátero Lagrangiano de Orden 2: <code>Quad9</code>	16
4.2.5.	Triángulo Cuadrático Completo P_2 : <code>Tri6</code>	16
5.	Catálogo de Modelos Constitutivos	17
5.1.	Elasticidad Lineal	17
5.1.1.	<code>Elastic1D</code> — elasticidad lineal axial	17
5.1.2.	<code>Elastic2D</code> — elasticidad lineal isótropa 2D	17
5.2.	Plasticidad	17
5.2.1.	<code>Elastoplastic1D</code> — plasticidad J2 axial con endurecimiento isótropo	17
5.2.2.	<code>VonMises2D</code> — plasticidad J2 plana con endurecimiento isótropo	18
5.2.3.	<code>DruckerPrager2D</code> — plasticidad friccional cohesivo-friccional 2D	19
5.3.	Daño Mecánico	20
5.3.1.	<code>IsotropicDamage1D</code> — daño isótropo 1D con softening exponencial	20
5.3.2.	<code>IsotropicDamage2D</code> — daño isótropo 2D	20
5.4.	Elasticidad Unilateral (Cable)	21
5.4.1.	<code>CableMaterial1D</code> — cable lineal en tensión, nulo en compresión	21
6.	Esquemas de Solución	22
6.1.	<code>LinearSolver</code> — solución directa lineal	22
6.2.	<code>NonlinearSolver</code> — Newton-Raphson incremental con paso adaptativo	22
6.3.	<code>ArcLengthSolver</code> — longitud de arco cilíndrico (Crisfield)	23
7.	Análisis Dinámico	24
7.1.	<code>ModalSolver</code> — análisis modal por autovalores generalizados	24
7.2.	<code>NewmarkSolver</code> — transitorio lineal Newmark- β	25
7.3.	<code>NewtonNewmarkSolver</code> — transitorio no lineal Newton-Newmark	26
8.	Post-Procesamiento y Visualización	27
8.1.	Formato VTU (VTK XML)	27
8.1.1.	Datos Nodales (Point Data)	27
8.1.2.	Datos de Elemento (Cell Data)	27
8.2.	Animación con Archivo PVD	27
8.3.	Salida en Texto Plano (Estilo FEAP)	27
8.4.	Workflow ParaView	28
9.	Ejemplos Completos	29
9.1.	Marco 2D — Viga en Voladizo	29
9.2.	Placa Continua con Plasticidad J2 (Gmsh)	30
9.3.	Cable Pretensado en Régimen Corotacional	31
10.	Uso Avanzado: API Programática	32
10.1.	Patrón Fachada	32

11.Diagnóstico de Problemas	33
-----------------------------	----

Sobre este manual

Este manual está dirigido al **usuario final** de Fenix FEM: cubre la sintaxis del archivo de entrada `.yaml`, el catálogo completo de elementos, materiales y solvers, ejemplos de uso y el workflow de post-procesamiento. No entra al detalle de la implementación interna ni a la formulación matemática completa de cada componente.

Para la *referencia formal* de cada componente (ecuaciones, matrices **B**, criterios de aceptación), consultar `manuals/Reference_manual.pdf`, generado automáticamente desde `docs/specs/`.

Para la *visión arquitectural* del programa (capas, bloques funcionales, mecanismos transversales y dirección de evolución), consultar `manuals/Architecture_manual.pdf`.

Este manual se regenera con:

```
python manuals/build_user_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los archivos fuente en `manuals/sources/user/`.

Capítulo 1

Introducción y Configuración del Entorno

1.1 Filosofía del Proyecto

Fenix FEM nace como herramienta de investigación en mecánica del medio continuo y análisis no lineal de estructuras. Su diseño persigue dos objetivos simultáneos:

1. **Rigor numérico y físico.** Cada formulación incorporada está validada contra solución analítica o benchmark establecido. La separación estricta **Elemento** $\leftrightarrow$ **Material** permite combinar cualquier cinemática con cualquier ley constitutiva siempre que sus protocolos sean compatibles.
2. **Crecimiento sin fricción.** La arquitectura está optimizada para que añadir un elemento, material o solver nuevo sea barato. Auto-registro vía decoradores, contratos declarativos (`STRAIN_DIM`, `DOF_NAMES`) y validación temprana del input son los mecanismos centrales.

1.1.1 Modelo Data-Driven

Toda la orquestación de un análisis se concentra en un archivo `.yaml` legible por humanos. El usuario no necesita escribir código Python para definir un modelo; la API programática queda reservada para casos avanzados (optimización paramétrica, mallas generadas proceduralmente, acoplamiento con otros sistemas).

1.2 Requisitos del Sistema

Fenix FEM se ejecuta sobre Python 3.8 o superior en Windows, Linux y macOS sin compilación nativa.

1.2.1 Dependencias

```
pip install numpy scipy pyyaml meshio numba
```

- **numpy** / **scipy**: álgebra lineal densa y dispersa; **spsolve** sobre matrices CSR.
- **pyyaml**: análisis sintáctico del archivo de entrada.

- **meshio**: importación de mallas Gmsh y exportación a VTK.
- **numba**: compilación JIT de los kernels críticos en elementos 2D y plasticidad.

1.3 Herramientas Externas Recomendadas

- **Gmsh**: generador de mallas; produce archivos `.msh` consumibles directamente desde el YAML.
- **ParaView**: visualización interactiva de los archivos `.vtu` generados.

1.4 Ejecución de un Análisis

El runner principal vive en `examples/ejecutar_yaml.py`. Para ejecutar un modelo:

```
python examples/ejecutar_yaml.py ruta/al/modelo.yaml
```

El script carga el YAML, valida su contenido (acumulando *todos* los errores en una sola pasada antes de abortar), construye el dominio, ensambla el solver pedido, ejecuta el análisis y exporta los resultados según el bloque `output`.

Capítulo 2

Arquitectura del Programa

2.1 Separación Elemento $\leftrightarrow$ Material

Cada elemento finito declara dos contratos sobre el material que acepta:

- **STRAIN_DIM**: dimensión Voigt de la deformación que entrega al material. 1 para elementos axiales (truss, cable, marcos — la fibra centroidal); 3 para 2D ($[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$); 6 para 3D continuo (no implementado todavía).
- **DOF_NAMES**: lista de grados de libertad por nodo. Por ejemplo, `['ux', 'uy', 'rz']` para un marco 2D.

El protocolo del material es uniforme:

```
sigma, E_tangent, new_state = material.compute_state(epsilon, state_vars)
```

donde `epsilon` tiene dimensión **STRAIN_DIM**, `sigma` es el esfuerzo en la misma representación, `E_tangent` es la matriz tangente consistente y `state_vars` encapsula las variables internas (deformación plástica, daño acumulado, etc.).

Un material declara igualmente su **STRAIN_DIM**; el dominio rechaza al construir cualquier emparejamiento incompatible.

2.2 Auto-Registro de Componentes

Elementos, materiales y solvers se registran automáticamente al arrancar el programa mediante decoradores:

```
@ElementRegistry.register
class Truss2D(Element):
    DOF_NAMES = ['ux', 'uy']
    STRAIN_DIM = 1
    ...
```

Esto permite que el archivo YAML los referencie por nombre (`type: Truss2D`) sin que el usuario tenga que importar nada manualmente. El módulo `fenix.autodiscover` recorre los paquetes `elements/`, `materials/` y `math/solvers/` en la primera importación y puebla los registries.

2.3 Validación Temprana del Input

El parser YAML acumula todos los errores detectados en una sola pasada (no aborta al primero) y los presenta juntos. Comprueba:

- Ausencia de bloques obligatorios (`nodes`, `materials`, `elements`, `solver`).
- IDs duplicados de nodo, material o elemento.
- Referencias a materiales o nodos inexistentes desde `elements` y `boundary_conditions`.
- Tipos de elemento, material y solver no registrados.
- **Parámetros no aceptados por el constructor del elemento.** Si un elemento define `A` e `I` como parámetros, escribir `large_strains: true` en su entrada YAML produce un error explícito que indica los parámetros válidos para ese tipo.

Capítulo 3

Archivo de Entrada YAML

La interacción exclusiva con el motor de Fenix FEM se realiza mediante el archivo de entrada `.yaml`. Un archivo válido contiene los bloques siguientes (algunos opcionales según el caso de uso):

- `nodes` o `mesh` — geometría.
- `materials` — catálogo de leyes constitutivas instanciadas.
- `elements` — conectividad (omisible si se importa malla Gmsh).
- `boundary_conditions_by_node` / `_by_coord` / `_by_group` — restricciones de Dirichlet.
- `point_loads_by_node` / `_by_coord` / `_by_group` — cargas de Neumann puntuales.
- `solver` — tipo de solver y sus parámetros.
- `output` — frecuencia y formato de exportación de resultados.

3.1 Geometría: `nodes` y `elements`

Para modelos pequeños o estructuras reticulares (armaduras, marcos, cables), la geometría se escribe directamente en el YAML:

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}

elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6
```

Las coordenadas pueden ser de 2 o 3 componentes; los elementos 1D 3D (`Truss3D`, `Cable3DCorot`, `Frame3D`) requieren las tres.

3.2 Importación de Malla Gmsh: mesh

Para problemas continuos 2D, se sustituye el bloque `nodes/elements` por una referencia al archivo `.msh`:

```
mesh: "geometria_placa.msh"
mesh_material: 1
mesh_thickness: 0.1
mesh_quadrature: "2x2"
```

Parámetro	Descripción
<code>mesh</code>	Ruta al archivo <code>.msh</code> de Gmsh, relativa al YAML.
<code>mesh_material</code>	ID del material por defecto que se asocia a toda la malla.
<code>mesh_thickness</code>	Espesor por defecto (estado plano 2D).
<code>mesh_quadrature</code>	Regla de Gauss: "2x2" (completa, recomendada) o "1x1" (reducida; produce <i>hourglassing</i>).

3.2.1 Mapeo Avanzado: mesh_physical_groups

Si la geometría tiene zonas con materiales o espesores distintos definidos como *Physical Groups* en Gmsh, se mapean explícitamente:

```
mesh_physical_groups:
  "Zona_Hormigon":
    material: 1
    thickness: 0.20
    quadrature: "2x2"
  "Zona_Acero":
    material: 2
    thickness: 0.05
```

3.3 Materiales

Cada material se declara con un `id` único y un `type` registrado en el catálogo:

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_strain
  density: 7850.0
```

Los parámetros aceptados dependen del material; consulte el capítulo *Catálogo de Modelos Constitutivos* para el catálogo completo.

3.4 Condiciones de Frontera

Fenix FEM ofrece tres mecanismos para imponer restricciones cinemáticas (Dirichlet), elegibles según la facilidad operativa:

3.4.1 Por nodo: `boundary_conditions_by_node`

```
boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}
- {node_id: 4, ux: 0.0, uy: 0.0}
```

3.4.2 Por coordenada: `boundary_conditions_by_coord`

Escanea todos los nodos y aplica la restricción a aquellos cuya coordenada coincide (dentro de tol):

```
boundary_conditions_by_coord:
  x_min: {ux: 0.0, uy: 0.0, tol: 1.0e-5}      # empotrar lado izquierdo
  rodillos: {coord: 'y', val: 10.0, uy: 0.0, tol: 1.0e-4}
```

Las claves predefinidas `x_min`, `x_max`, `y_min`, `y_max` usan los extremos detectados automáticamente del dominio.

3.4.3 Por grupo físico: `boundary_conditions_by_group`

Aplica restricciones a todos los nodos de un *Physical Group* de Gmsh:

```
boundary_conditions_by_group:
  "Borde_Empotrado": {ux: 0.0, uy: 0.0}
```

3.4.4 Restricciones multipunto (MPC): `linear_constraints`

Para apoyos en plano oblicuo, periodicidad de celda unitaria, uniones rígidas master-slave y simetrías no alineadas con los ejes globales, Fenix FEM admite restricciones afines lineales de la forma $u_s = g_s + \sum_i \alpha_{si} u_{m_i}$ (ADR 0004 fase 2). Se declaran en el bloque `linear_constraints`:

```
linear_constraints:
# Apoyo a 45 grados en el nodo 4: u_y = u_x.
- slave: {node: 4, dof: uy}
  masters:
    - {node: 4, dof: ux}
  coefficients: [1.0]

# Periodicidad: u_x del nodo 7 igual al del nodo 3.
- slave: {node: 7, dof: ux}
  masters:
    - {node: 3, dof: ux}
  coefficients: [1.0]

# Union rigida: u_x del satellite N9 sigue al maestro N5 con offset r_y=0.5.
- slave: {node: 9, dof: ux}
  masters:
    - {node: 5, dof: ux}
    - {node: 5, dof: rz}
```

```
coefficients: [1.0, -0.5]
g: 0.0
```

Las restricciones se imponen por eliminación directa (no por penalización), de modo que la imposición es exacta a redondeo y preserva la simetría y la positividad definida del sistema. Las cadenas master-slave se resuelven automáticamente por cierre transitivo; los ciclos y las redeclaraciones inconsistentes se detectan en construcción y abortan el análisis con un mensaje específico.

3.5 Cargas Puntuales

Misma estructura que las condiciones de frontera, en bloques `point_loads_by_node`, `point_loads_by_coord` y `point_loads_by_group`. Los valores son fuerzas nodales (en las unidades del modelo, típicamente Newtons):

```
point_loads_by_node:
  - {node_id: 3, uy: -15000.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}
```

3.6 Fuerzas de Cuerpo

Una *fuerza de cuerpo* es una fuerza por unidad de volumen $\mathbf{b}$ que actúa sobre todo el dominio material del elemento, no solo sobre su frontera. Cada elemento del catálogo (armaduras, cables, marcos 2D/3D, sólidos 2D) integra la contribución consistente $\int \mathbf{N}^T \mathbf{b} A dx$ y la acumula al vector global de fuerzas. Para vigas Hermite cúbicas el reparto incluye fuerza nodal $qL/2$ y momento $\pm qL^2/12$; para barras axiales es mitad-mitad por nodo.

Casos típicos en mecánica estructural: peso propio (un caso particular con $\mathbf{b} = \rho \mathbf{g}$), fuerzas centrífugas en cuerpos rotatorios ($\mathbf{b} = \rho \omega^2 \mathbf{r}$), fuerzas electromagnéticas sobre materiales conductores, expansión térmica modelada como pseudocarga. Fenix expone dos formas declarativas en YAML que cubren todos estos casos.

3.6.1 Forma general: `body_force`

Aplica un vector $\mathbf{b}$ uniforme (fuerza por unidad de volumen, en ejes globales) a todos los elementos del modelo:

```
body_force: [0.0, -77008.5] # 2D: bx, by en N/m^3
```

o, para problemas 3D:

```
body_force: [0.0, 0.0, -77008.5] # 3D: bx, by, bz
```

Es la forma genérica y la única necesaria para fuerzas de cuerpo que no son peso propio (campos electromagnéticos precalculados, centrífuga estática, etc.). También sirve para peso propio cuando todo el modelo es monomaterial y el usuario prefiere precalcular $\rho \cdot g$.

3.6.2 Caso particular — peso propio: `gravity + density`

El peso propio es el caso especial $\mathbf{b} = \rho \mathbf{g}$. Para problemas multimaterial — donde cada elemento debe ver *su propio* ρ — Fenix expone un atajo declarativo: la densidad como propiedad del material, y un vector global de gravedad.

```
materials:
  - {id: 1, type: Elastic1D, E: 210.0e9, density: 7850.0}      # acero
  - {id: 2, type: Elastic1D, E: 30.0e9, density: 2500.0}      # hormigón

gravity: [0.0, -9.81]                                         # m/s^2, +y hacia arriba
```

Qué hace Fenix. Para cada elemento computa $\mathbf{b}_e = \rho_e \cdot \mathbf{g}$, donde ρ_e es la densidad del material asignado, e integra la contribución consistente con la misma maquinaria de `body_force`. Resultado: estructuras multimaterial dan peso propio físicamente correcto, cada elemento con su propio ρ .

Densidad solo cuando se usa. El campo `density` es **opcional** al declarar un material (ADR 0008): los análisis estáticos puros que no invocan peso propio ni matriz de masa no necesitan declararla. Pero al invocar `gravity` (o, en su día, análisis modal/dinámico), Fenix exige la densidad de cada material involucrado: si algún material no la trae declarada, el cálculo falla con `ValueError` listando los materiales afectados por nombre. Imposible propagar masa cero silenciosa. Si un material legítimamente carece de masa física (penalty, restricción), declararlo explícitamente como `density: 0.0` — caso en el cual `gravity` emite un `WARNING` informativo y su contribución al peso propio es nula.

3.6.3 Detalles transversales

Exclusividad. `body_force` y `gravity` son dos formas declarativas para expresar fuerzas de cuerpo; declarar ambas en el mismo archivo lanza error de validación. La forma general (`body_force`) es la subyacente; `gravity` es atajo para su caso particular más común.

Padding 2D↔3D. Si el modelo mezcla elementos 2D y 3D, declarar el vector con tres componentes; el ensamblador pasa el slice apropiado a cada elemento según la dimensionalidad declarada por su `DOF_NAMES`.

Reservado para análisis dinámico. El atributo `density` introducido aquí es la misma magnitud que consume la matriz de masa $\mathbf{M}_e = \int \rho \mathbf{N}^\top \mathbf{N} dV$ en análisis modales y dinámicos. Declararlo prepara el modelo para esas extensiones sin coste adicional (ADR 0008).

3.7 Solver

```
solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5
```

Los solvers disponibles y sus parámetros se documentan en los capítulos *Esquemas de Solución* y *Análisis Dinámico*.

3.7.1 Bloque convergence

Los solvers no lineales (`NonlinearSolver`, `ArcLengthSolver`) aceptan un bloque `convergence` que configura el criterio de parada de las iteraciones de Newton. La política sigue el patrón estándar de tolerancias mixtas absoluta + relativa, separado por dos criterios físicos: *fuerza* (residuo de equilibrio) y *desplazamiento* (incremento del iterado). La forma exacta de la comparación es:

```
||R[libres]|| <= atol_force + rtol_force * max(||F_ext||, ||F_int||)
||dU|| <= atol_disp + rtol_disp * ||U||
```

Ambos criterios deben cumplirse **simultáneamente** (semántica AND) para dar el paso por convergido.

Parámetros disponibles (todos opcionales, con defaults razonables):

- **rtol_force** (default 1.0e-5): tolerancia relativa del residuo de fuerza. Es el parámetro que normalmente ajustarás: bajarlo a 1.0e-7 para análisis de precisión, subirlo a 1.0e-3 para exploración rápida.
- **rtol_disp** (default 1.0e-5): tolerancia relativa de la corrección de desplazamiento. En problemas casi-rígidos puede afinarse independientemente del de fuerza.
- **atol_force_factor** (default 1.0e-9): factor adimensional que define el piso absoluto de fuerza. La tolerancia absoluta efectiva se autoderiva como **atol_force_factor** · **escala_inicial**, donde la escala se calcula en el primer ensamblaje. Solo conviene ajustarlo si el análisis tiene tramos donde la carga característica colapsa transitoriamente y aparece falsa no-convergencia.
- **atol_disp_factor** (default 1.0e-9): análogo para desplazamiento.

Por qué los atol se autoderivan. El término absoluto vive en las unidades del problema (newtons para fuerza, metros para desplazamiento). Codificarlo como constante global obligaría a ajustarlo al cambiar de unidades (N/m, kN/mm, MPa/mm...). En su lugar, Fenix calcula la escala característica del problema en el primer ensamblaje y multiplica por el factor adimensional. El resultado: **el mismo análisis planteado en distintos sistemas de unidades converge en el mismo número de iteraciones hasta paridad de bits**, sin tocar las tolerancias. La justificación arquitectural está en el ADR 0007.

Cuándo afinar. Para la inmensa mayoría de análisis los defaults son suficientes. Considera ajustar:

- **rtol_force** y **rtol_disp** más estrictos cuando publiques resultados o compares contra solución analítica (1.0e-7 a 1.0e-9).
- **rtol_disp** más laxo que **rtol_force** en problemas casi-rígidos donde el incremento de desplazamiento es naturalmente pequeño y baja muy rápido, dejando que el criterio de fuerza domine.
- Los factores **atol_*** prácticamente nunca; solo si encuentras un análisis específico donde la escala colapsa y los logs muestran iteraciones oscilando cerca del umbral.

Nota sobre el solver algebraico interno. El sistema lineal $\mathbf{K} \delta \mathbf{U} = \mathbf{R}$ que aparece dentro de cada iteración se resuelve con un backend algebraico (Cholesky, LU...) seleccionado *automáticamente* según las propiedades de $\mathbf{K}$. Esta elección **no** es decisión de modelado y no requiere configuración. Si por motivos de diagnóstico necesitas forzar un backend específico, consulta el capítulo *Anexos técnicos — Capa algebraica* del Manual de Referencia.

3.8 Salida: output

Configura qué se exporta y cuándo:

```
output:
  file_name: "resultados_placa"
  pre_process: {export: true}      # exporta el modelo sin deformar (paso 0)
  results:
    frequency: "all_steps"        # "last_step" o "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State", "Sigma_XX"]
  text_export:                    # opcional: salida estilo FEAP en .txt
    export: true
    nodes: all
    elements: all
```

Nota

Cuando frequency: ".all_steps" y el solver es no lineal, se genera un archivo .vtu por paso convergido más un archivo maestro .pvd que ParaView interpreta como animación.

Capítulo 4

Catálogo de Elementos Finitos

El motor expone dos familias de elementos: **1D** (estructuras reticulares: armaduras, cables, marcos) y **2D** (continuo plano: cuadrilátero y triángulo). Todos se referencian desde el YAML por su nombre exacto en `type:`.

4.1 Elementos 1D

4.1.1 Armaduras: `Truss2D` / `Truss2DCorot` / `Truss3D` / `Truss3DCorot`

Barra biarticulada que transmite exclusivamente esfuerzo axial. Cuatro variantes según dimensión y régimen geométrico:

Tabla 4.1: Familia de armaduras.

Elemento	DOFs/nodo	Dimensión	Régimen geom.	Uso
Truss2D	ux, uy	2D	lineal	cargas pequeñas, rotaciones pequeñas
Truss2DCorot	ux, uy	2D	corotacional	grandes desplazamientos y rotaciones planas
Truss3D	ux, uy, uz	3D	lineal	armaduras espaciales en régimen lineal
Truss3DCorot	ux, uy, uz	3D	corotacional	grandes rotaciones en el espacio

Parámetros: A (área de la sección transversal). Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).

Régimen de validez: $|\varepsilon| \lesssim 10^{-2}$. Las variantes `Corot` aceptan rotaciones de cualquier magnitud entre commits; las versiones lineales asumen rotaciones pequeñas.

```
elements:
- id: 1
  type: Truss2DCorot
  material: 1
  nodes: [1, 2]
  A: 0.0005
```

4.1.2 Cables: Cable2DCorot / Cable3DCorot

Elemento 1D que transmite *únicamente* esfuerzo de tensión (no resiste compresión). La unilateralidad la aporta el material (típicamente `CableMaterial1D`, ver capítulo *Catálogo de Modelos Constitutivos*); el elemento implementa la cinemática corotacional y la transferencia de la deformación al material.

- **DOFs/nodo:** u_x, u_y (2D); u_x, u_y, u_z (3D).
- **Parámetros:** A .
- **Cinemática:** corotacional (longitud y cosenos directores se recalculan en cada evaluación).
- **Régimen de validez:** $|\varepsilon| \lesssim 10^{-2}$ tensado. En estado destensado el elemento aporta rigidez nula al sistema global; otros elementos deben garantizar la estabilidad numérica.

Advertencia

Un cable completamente destensado ($\sigma = 0$) tiene $\mathbf{K}_T = \mathbf{0}$ en sus DOFs. Si todos los caminos de carga del modelo dependen del cable y este se destensa, la matriz tangente global se vuelve singular. Pretensar el cable o garantizar redundancia estructural.

```
materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9
  density: 7850.0

elements:
- id: 1
  type: Cable2DCorot
  material: 1
  nodes: [1, 2]
  A: 5.0e-5
```

4.1.3 Marcos 2D: Frame2DEuler / Frame2DTimoshenko / Frame2DEulerCorot

Elementos viga 2D que transmiten axial, cortante y momento flector. Tres variantes:

- **Frame2DEuler:** vigas esbeltas ($L/h \gtrsim 10$), Euler-Bernoulli, régimen geoméricamente lineal. Parámetros: A, I .
- **Frame2DTimoshenko:** vigas peraltadas o cortas ($L/h \lesssim 10$). Incluye deformación por cortante; corrige automáticamente *shear locking* en el límite esbelto. Parámetros: A, I, A_s (área efectiva de cortante), ν (opcional si el material expone ν).
- **Frame2DEulerCorot:** variante corotacional de Euler-Bernoulli. Captura grandes desplazamientos y grandes rotaciones rígidas del elemento (rotaciones deformacionales nodales moderadas, $|\bar{\theta}| \lesssim 30^\circ$). Parámetros: A, I .

DOFs/nodo: u_x, u_y, r_z . Convención: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación) en el eje axial; r_z positivo antihorario.

Advertencia

Plasticidad por flexión. Los elementos marco pasan al material únicamente la deformación axial centroidal $(u_j - u_i)/L$; el módulo tangente E_t devuelto escala toda la matriz local (axial y flexión por igual). Esto significa que los marcos reproducen correctamente *plasticidad axial pura* pero *no* forman rótulas plásticas por flexión: la fluencia por momento requiere integración de $\sigma(y)$ sobre la sección, característica que se incorporará en una clase **FiberSection** futura.

```
elements:
- id: 1
  type: Frame2DEulerCorot
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833
```

4.1.4 Marco 3D: Frame3D

Viga 3D Euler-Bernoulli con 6 DOFs/nodo ($u_x, u_y, u_z, r_x, r_y, r_z$). Transmite axial, cortantes en dos planos, flexiones en dos planos y torsión de Saint-Venant pura. Régimen geoméricamente lineal (la variante corotacional 3D queda como pendiente).

Parámetros:

- **A** — área de la sección.
- **I_y, I_z** — momentos de inercia respecto a los ejes locales y, z .
- **J** — constante torsional de Saint-Venant.
- **nu** — coeficiente de Poisson (opcional; se toma del material si lo expone).
- **ref_vector** — vector de referencia (opcional, default $[0,0,1]$) que fija la orientación de los ejes locales y, z de la sección. Para barras casi-verticales se sugiere fijarlo explícitamente.

```
elements:
- id: 1
  type: Frame3D
  material: 1
  nodes: [1, 2]
  A: 0.01
  Iy: 8.33e-6
  Iz: 8.33e-6
  J: 1.66e-5
  ref_vector: [0.0, 0.0, 1.0]
```

4.2 Elementos 2D

La importación de mallas desde Gmsh instancia automáticamente los elementos 2D según la topología detectada (cuadriláteros $\rightarrow$ **Quad4**; triángulos $\rightarrow$ **Tri3**). También pueden declararse manualmente desde el bloque `elements`.

4.2.1 Cuadrilátero Bilineal: Quad4

Elemento isoparamétrico de 4 nodos, integración Gauss configurable.

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM = 3`, Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$).
- **Nodos:** 4, ordenados en sentido antihorario.
- **Cuadratura:** "2x2" por defecto (4 puntos). "1x1" disponible pero produce modos espurios (*hourglassing*).
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Implementación:** kernels críticos compilados con `@njit` (Numba).
- **Limitación:** bloqueo volumétrico con materiales casi-incompresibles ($\nu \rightarrow 0,5$); en ese régimen requeriría formulación mixta (no implementada).

4.2.2 Triángulo de Deformación Constante: Tri3

Triángulo CST de 3 nodos, un único punto de integración.

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM = 3`).
- **Cuadratura:** 1 punto (deformación uniforme).
- **Parámetros:** `thickness`.
- **Limitación:** *shear locking* severo, convergencia lenta. **Preferir Quad4** salvo en transiciones donde Quad4 no encaja geométricamente.

4.2.3 Cuadrilátero Serendípito de Orden 2: Quad8

Cuadrilátero isoparamétrico cuadrático de 8 nodos (4 vértices antihorarios + 4 nodos medios de borde). Funciones de forma serendípitas (sin el término $\xi^2\eta^2$). Reproduce campos cuadráticos exactamente y mitiga el bloqueo por cortante en problemas de flexión.

- **DOFs/nodo:** `ux`, `uy` (`STRAIN_DIM = 3`).
- **Cuadratura:** Gauss 3×3 (9 puntos) por defecto; 2×2 disponible vía `quadrature` pero con riesgo de modos espurios.
- **Parámetros:** `thickness`, `quadrature` (opcional).
- **Tracción de borde:** reparte $1/6$, $4/6$, $1/6$ entre los nodos del borde (vértice, medio, vértice).

4.2.4 Cuadrilátero Lagrangiano de Orden 2: Quad9

Cuadrilátero Lagrangiano cuadrático de 9 nodos (los 8 de Quad8 más un noveno nodo central interior). Las funciones de forma son producto tensorial Lagrange 1D-1D, espacio polinómico completo Q_2 .

- **DOFs/nodo:** u_x, u_y (STRAIN_DIM = 3).
- **Cuadratura:** Gauss 3×3 .
- **Parámetros:** thickness, quadrature (opcional).
- **Diferencia con Quad8:** el nodo central interior añade el término $\xi^2 \eta^2$ y mejora el comportamiento en problemas con campos no separables.

4.2.5 Triángulo Cuadrático Completo P_2 : Tri6

Triángulo isoparamétrico cuadrático de 6 nodos (3 vértices + 3 medios de borde). Resuelve el *shear locking* severo de Tri3 y reproduce campos cuadráticos exactamente.

- **DOFs/nodo:** u_x, u_y (STRAIN_DIM = 3).
- **Cuadratura:** 3 puntos en los puntos medios (regla `tri_3`).
- **Parámetros:** thickness.
- **Tracción de borde:** reparte 1/6, 4/6, 1/6 entre los nodos del borde (vértice, medio, vértice).
- **Cuándo usarlo:** transiciones de malla cuadráticas, geometrías curvadas donde Quad8/Quad9 no encajan.

```
elements:
- {id: 1, type: Quad8, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8]}
- {id: 2, type: Quad9, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6, 7, 8, 9]}
- {id: 3, type: Tri6, material: 1, thickness: 0.1, nodes: [1, 2, 3, 4, 5, 6]}
```

Capítulo 5

Catálogo de Modelos Constitutivos

5.1 Elasticidad Lineal

5.1.1 Elastic1D — elasticidad lineal axial

Ley de Hooke escalar $\sigma = E \cdot \varepsilon$. Sin variables internas. Compatible con todos los elementos 1D (armaduras y marcos).

```
materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0
```

5.1.2 Elastic2D — elasticidad lineal isótropa 2D

Tensor constitutivo isótropo $\sigma = \mathbf{C} \cdot \varepsilon$ en notación Voigt $[xx, yy, xy]$. Sin variables internas. Compatible con Quad4, Tri3.

```
materials:
- id: 1
  type: Elastic2D
  E: 200.0e9
  nu: 0.3
  hypothesis: plane_stress
  density: 7850.0
```

La hipótesis `plane_stress` se usa para placas delgadas sin confinamiento normal; `plane_strain` para secciones de presa, túneles o problemas con extensión infinita en el eje longitudinal.

5.2 Plasticidad

5.2.1 Elastoplastic1D — plasticidad J2 axial con endurecimiento isótropo

Plasticidad asociativa con criterio $f = |\sigma_{\text{trial}}| - (\sigma_y + H\alpha) \leq 0$. Algoritmo de *return mapping* clásico 1D; tangente algorítmica consistente $E_t = E H / (E + H)$.

- **Parámetros:** E, `sigma_y` (fluencia inicial), H (módulo de endurecimiento; H=0 = perfectamente plástico).

- **Variables internas:** ε_p (deformación plástica), α (acumulada equivalente).
- **Compatible con:** armaduras y marcos 1D (en marcos solo se aplica al esfuerzo axial; ver advertencia en el capítulo *Catálogo de Elementos Finitos*).

```
materials:
- id: 1
  type: Elastoplastic1D
  E: 200.0e9
  sigma_y: 250.0e6
  H: 2.0e9
  density: 7850.0
```

5.2.2 VonMises2D — plasticidad J2 plana con endurecimiento isótropo

Plasticidad asociativa con criterio J2 y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes, seleccionadas con el campo `hypothesis` al construir el material:

- `plane_strain` ($\varepsilon_{zz} = 0$): return mapping radial cerrado sobre la parte desviadora 3D extendida (Simó-Hughes §3.3). Corrector $\Delta\gamma = f_{\text{trial}}/(2G + \frac{2}{3}H)$ con $N = s_{\text{trial}}/\|s_{\text{trial}}\|$ y actualización $\alpha_{\text{new}} = \alpha + \sqrt{2/3}\Delta\gamma$. Default histórico.
- `plane_stress` ($\sigma_{zz} = 0$): *plane stress projected algorithm* (Simó-Hughes §3.4.1). Función de fluencia proyectada $\bar{f} = \frac{1}{2}\sigma^\top \mathbf{P} \sigma - R^2/3$ y Newton local escalar sobre $\Delta\gamma$. Converge en 3–6 iteraciones por evaluación. Actualización físicamente correcta $\alpha_{\text{new}} = \alpha + \Delta\gamma\sqrt{2\sigma^\top \mathbf{P} \sigma}/3$ (en plane stress $\Delta\gamma$ tiene unidades [1/esfuerzo], por lo que la heurística $\sqrt{2/3}\Delta\gamma$ válida en plane strain rompe la invariancia bajo cambio de unidades). Incompresibilidad plástica cierra $e_{zz}^p = -(e_{xx}^p + e_{yy}^p)$.

Parámetros comunes: E, nu, sigma_y, H (≥ 0 , default 0 = perfectamente plástico), `hypothesis` (default `plane_strain`), `density` (opcional). Variables internas: ε_p tensorial 4 componentes $[xx, yy, zz, xy_{\text{tens}}]$ y α acumulada equivalente adimensional. **Tangente algorítmica consistente cerrada** en ambas hipótesis (con corrección por $d\alpha/d\Delta\gamma \neq \sqrt{2/3}$ en plane stress). Compatible con todos los elementos 2D del catálogo.

```
materials:
# Hipótesis por defecto: plane_strain
- id: 2
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
  H: 2.0e9
  hypothesis: plane_strain
  density: 7850.0

# Mismo material en plane_stress (placas delgadas sin confinamiento normal)
- id: 3
  type: VonMises2D
  E: 210.0e9
  nu: 0.3
  sigma_y: 250.0e6
```

```
H: 2.0e9
hypothesis: plane_stress
density: 7850.0
```

5.2.3 DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D

Modelo de Drucker-Prager: cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio de fluencia $f = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$ con $k(\alpha) = k_0 + H\alpha$ (endurecimiento isótropo lineal en cohesión). Plasticidad *no* asociada por defecto: el ángulo de dilatación ψ es parámetro independiente del ángulo de fricción ϕ .

Algoritmo — dos ramas con detección automática:

- **Return regular** (superficie del cono): $\Delta\gamma = f_{\text{trial}}/(G + 9K\eta_f\eta_g + H)$, dirección desviadora preservada. Tangente algorítmica $\mathbf{C}_{\text{alg}} = K\mathbf{v} \otimes \mathbf{v} + 2G(1 - \beta)\mathbf{I}_{\text{dev}} + 4G\beta\hat{\mathbf{n}} \otimes \hat{\mathbf{n}} - (1/A)\mathbf{b}_g \otimes \mathbf{b}_f$ con $\beta = G\Delta\gamma/\sqrt{J_2^{\text{trial}}}$.
- **Return al ápice** (vértice puramente hidrostático): si tras el return regular el estado caería fuera del cono, conmuta a $\Delta\gamma_{\text{apex}} = (I_1^{\text{trial}}\eta_f - k(\alpha))/(9K\eta_f\eta_g + H)$ con $\sigma_{n+1} = (k/3\eta_f)\mathbf{I}$ puramente hidrostático. Tangente reducida $KH/(9K\eta_f\eta_g + H)\mathbf{v} \otimes \mathbf{v}$.

Calibración con Mohr-Coulomb (campo variant):

- **plane_strain_matched** (default): coincide exactamente con MC en plane strain. $\eta_f = \tan\phi/\sqrt{9 + 12\tan^2\phi}$, $k_0 = 3c_0/\sqrt{9 + 12\tan^2\phi}$.
- **outer_cone**: circumscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 - \sin\phi)]$.
- **inner_cone**: inscribe MC. $\eta_f = 2\sin\phi/[\sqrt{3}(3 + \sin\phi)]$.

Parámetros: E, nu, cohesion (cohesión inicial c_0), phi_deg ($0 \leq \phi < 90$), psi_deg (dilatación, $0 \leq \psi \leq \phi$; default $\psi = \phi$ = asociada), H (≥ 0 , default 0), variant (default plane_strain_matched), density (opcional). Variables internas: ε_p tensorial 4 componentes y α acumulada. IS_SYMMETRIC = False declarativo (no asociada $\Rightarrow$ tangente asimétrica; el despachador algebraico ADR 0003 elige LU).

Advertencia

Restricción cinemática: solo plane_strain. La proyección con $\sigma_{zz} = 0$ acoplada a flujo dilatante es notoriamente delicada y queda *out-of-scope* en esta entrega.

```
materials:
- id: 4
  type: DruckerPrager2D
  E: 30.0e9
  nu: 0.25
  cohesion: 2.0e5      # Pa
  phi_deg: 30.0
  psi_deg: 10.0       # no asociada (psi < phi)
  H: 0.0              # plasticidad perfecta
  variant: plane_strain_matched
  density: 2000.0     # arena densa
```


5.3 Daño Mecánico

5.3.1 IsotropicDamage1D — daño isótropo 1D con softening exponencial

Daño escalar $d \in [0, 1)$ con esfuerzo nominal $\sigma = (1 - d) E \varepsilon$. Evolución por norma de la deformación equivalente $\varepsilon_{eq} = |\varepsilon|$ y umbral κ_0 :

$$d = 1 - \frac{\kappa_0}{\kappa} \exp(-\alpha(\kappa - \kappa_0)) \quad \text{para } \kappa > \kappa_0$$

- **Parámetros:** `E`, `kappa_0` (umbral elástico), `alpha` (velocidad de softening), `density` (opcional).
- **Variables internas:** κ (máxima histórica), d (daño).
- **Tangente: algorítmica consistente** en carga activa con daño no saturado: $E_{\tan} = -(1 - d) E \alpha \kappa$ — negativa, refleja la pendiente descendente σ - ε post-pico. En descarga, sin daño activo o al saturar, vuelve a la secante $(1 - d)E$.

5.3.2 IsotropicDamage2D — daño isótropo 2D

Extensión 2D del modelo anterior. Esfuerzo nominal $\boldsymbol{\sigma} = (1 - d) \mathbf{C}_e \boldsymbol{\varepsilon}$. Deformación equivalente simétrica:

$$\varepsilon_{eq} = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2} \gamma_{xy}^2}$$

- **Parámetros:** `E`, `nu`, `kappa_0`, `alpha`, `hypothesis` (`plane_stress` default o `plane_strain`), `density` (opcional).
- **Tangente algorítmica consistente:** $\mathbf{C}_{\text{alg}} = (1 - d) \mathbf{C}_e - \frac{(1-d)(1/\kappa+\alpha)}{\varepsilon_{eq}} (\mathbf{C}_e \boldsymbol{\varepsilon}) \otimes (\mathbf{M} \boldsymbol{\varepsilon})$ con $\mathbf{M} = \text{diag}(1, 1, 1/2)$ en carga activa. **No simétrica** ($\mathbf{C}_e \boldsymbol{\varepsilon}$ y $\mathbf{M} \boldsymbol{\varepsilon}$ no son proporcionales); el atributo `IS_SYMMETRIC = False` hace que el despachador algebraico ADR 0003 elija LU. Recupera convergencia cuadrática del Newton global frente a la secante.
- **Limitación:** ε_{eq} simétrica no distingue daño en tensión vs compresión; para hormigón se requiere modelo de Mazars o split tensión/compresión (no implementado). Sin regularización por longitud característica $\rightarrow$ *mesh-dependency* en régimen de ablandamiento.

```
materials:
- id: 3
  type: IsotropicDamage2D
  E: 30.0e9
  nu: 0.2
  kappa_0: 1.5e-4
  alpha: 800.0
  hypothesis: plane_stress
  density: 2500.0 # hormigón
```

5.4 Elasticidad Unilateral (Cable)

5.4.1 CableMaterial1D — cable lineal en tensión, nulo en compresión

Material 1D *memoryless* que captura la propiedad esencial del cable: solo transmite esfuerzo cuando está tensado.

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E \varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

con $\langle \cdot \rangle^+$ el operador de MacAuley (parte positiva). Tangente discontinua en $\varepsilon = 0$ (asignación al régimen compresivo por convención).

- **Parámetros:** E (módulo de Young en tensión).
- **Sin variables internas** (la respuesta depende sólo de ε instantánea).
- **Caveats numéricos:** en transiciones tensado $\leftrightarrow$ destensado Newton-Raphson puede oscilar. Mitigación: usar `ArcLengthSolver` o pasos finos con `adaptive`.

```
materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9
  density: 7850.0
```

Capítulo 6

Esquemas de Solución

6.1 LinearSolver — solución directa lineal

Resuelve $\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$ en un único `spsolve`. Las condiciones de Dirichlet se imponen por eliminación directa (ADR 0004): el sistema entregado al backend algebraico es ya el reducido $\mathbf{K}_{\text{red}} \cdot \mathbf{U}_{\text{libre}} = \mathbf{F}_{\text{red}}$.

- **Cuándo usarlo:** problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **Parámetros:** `linear_algebra` (default `auto`; ADR 0003 §4).
- **No aplica:** cualquier no-linealidad material (daño, plasticidad, cable) o geométrica (corotacional). Producirá resultados inconsistentes.

```
solver:  
  type: LinearSolver
```

6.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

Bucle externo de incrementos del factor de carga $\lambda \in [0, 1]$; bucle interno Newton-Raphson sobre $\mathbf{K}_t \cdot \Delta \mathbf{U} = \mathbf{R} = \lambda \mathbf{F}_{\text{ext}} - \mathbf{F}_{\text{int}}$.

Criterio de convergencia dual:

$$\max(\text{err}_{\text{disp}}, \text{err}_{\text{force}}) < \text{tol}$$

con $\text{err}_{\text{disp}} = \|\Delta \mathbf{U}\| / (\|\mathbf{U}\| + \epsilon)$ y $\text{err}_{\text{force}} = \|\mathbf{R}\| / \max(\|\lambda \mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|, \epsilon)$.

Adaptatividad (`adaptive: true`):

- Si converge en menos de 5 iteraciones, agranda el siguiente paso ($\times 1,5$).
- Si no converge, biseca el paso ($\div 2$). Falla definitivamente si $\Delta \lambda < \text{min_delta_lambda}$.

```

solver:
  type: NonlinearSolver
  num_steps: 20
  max_iter: 15
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

Parámetros: convergence (bloque con rtol_force, rtol_disp, atol_force_factor, atol_disp_factor; ver ADR 0007), max_iter, num_steps, adaptive, min_delta_lambda, linear_algebra, freeze_tangent_after_i

Limitación: no puede atravesar puntos límite (snap-through) ni recorrer ramas con derivada negativa de la curva carga-desplazamiento. Para esos casos → ArcLengthSolver.

6.3 ArcLengthSolver — longitud de arco cilíndrico (Crisfield)

Traza curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga λ .

Esquema:

- **Predictor tangente:** $\Delta \mathbf{U}_t = \mathbf{K}_t^{-1} \cdot \mathbf{F}_{\text{ext}}^{\text{ref}}$; $\Delta \lambda = \pm dl / \|\Delta \mathbf{U}_t\|$. Signo elegido por proyección con el incremento previo.
- **Corrector iterativo:** en cada iteración resuelve dos sistemas ($\mathbf{K} \cdot \Delta \mathbf{U}_R = \mathbf{R}$ y $\mathbf{K} \cdot \Delta \mathbf{U}_t = \mathbf{F}_{\text{ext}}$) y aplica la restricción cuadrática cilíndrica $\|\delta \mathbf{U}\|^2 = dl^2$.
- **Auto-ajuste de dl:** se agranda en convergencias rápidas, se reduce en lentas, biseca al fallar.
- **Caso especial final_step:** si el predictor sobrepasaría max_lambda, fija λ a ese valor y resuelve solo desplazamientos (Newton-Raphson puro).

```

solver:
  type: ArcLengthSolver
  max_iter: 15
  max_lambda: 1.0
  initial_dl: 0.05
  max_steps: 200
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

```

Cuándo usarlo: problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando NonlinearSolver diverge cerca de un punto límite. Más caro por paso (dos spsolve por iteración) pero indispensable en estos casos.

Referencia: Crisfield, ^A fast incremental/iterative solution procedure that handles snap-through” (*Computers & Structures*, 1981).

Capítulo 7

Análisis Dinámico

Fenix FEM expone tres análisis dinámicos cerrados por la fase 4 del ADR 0009: modal (autovalores generalizados), transitorio lineal (Newmark- β) y transitorio no lineal (Newton-Newmark). Todos consumen la matriz de masa consistente que cada elemento devuelve por `compute_mass_matrix(lumping=consistent)` y la densidad declarada en el material (`density`; obligatoria para cualquier análisis dinámico, ADR 0008).

7.1 ModalSolver — análisis modal por autovalores generalizados

Resuelve el problema generalizado $\mathbf{K}\phi = \omega^2\mathbf{M}\phi$ con $\mathbf{K}$ evaluada en $\mathbf{u} = 0$ (régimen lineal) y $\mathbf{M}$ ensamblada por cuadratura consistente sobre todos los elementos. La capa algebraica (`EigenSolver`) envuelve `scipy.sparse.linalg.eigsh` con *shift-invert* en σ (ARPACK Lanczos), de modo que basta una sola factorización de $\mathbf{K} - \sigma\mathbf{M}$ para extraer las n frecuencias más bajas.

Parámetros: `n_modes` (obligatorio), `sigma` (default 0.0; `sigma=0` extrae las frecuencias más bajas), `which` (default "LM"), `tolerance` (default 1.0e-9), `lumping` (default `consistent`); la masa concentrada queda diferida a fase 2 del ADR 0009).

Salida: instancia de `ModalResult` con `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (M-ortonormales). El método `ModalResult.free_vibration(M, u0, u0_dot, t)` reconstruye analíticamente la respuesta libre no amortiguada por superposición modal.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [1.0, 0.0]}

materials:
- {id: 1, type: Elastic1D, E: 200.0e9, density: 7850.0}

elements:
- {id: 1, type: Truss2D, material: 1, A: 0.01, nodes: [1, 2]}

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0}
- {node_id: 2, uy: 0.0}

solver:
  type: ModalSolver
  n_modes: 3
  sigma: 0.0
```

Nota

La densidad es *obligatoria* para todo material involucrado en análisis modal o transitorio. La omisión lanza **ValueError** (ADR 0008) listando los materiales afectados por nombre antes de iniciar el cálculo.

7.2 NewmarkSolver — transitorio lineal Newmark- β

Integra en el tiempo $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(t)$ por la familia Newmark- β . Con $(\beta, \gamma) = (1/4, 1/2)$ — *average acceleration*, default — es *incondicionalmente estable*, sin amortiguamiento numérico, con error $\mathcal{O}(\Delta t^2)$.

Esquema operativo:

- Predictor: $\tilde{\mathbf{u}} = \mathbf{u}_n + \Delta t \dot{\mathbf{u}}_n + (\Delta t^2/2)(1 - 2\beta)\ddot{\mathbf{u}}_n$ y análogo para $\tilde{\dot{\mathbf{u}}}$.
- Sistema efectivo en aceleración: $\mathbf{A}_{\text{eff}}\ddot{\mathbf{u}}_{n+1} = \mathbf{F}_{n+1} - \mathbf{C}\tilde{\dot{\mathbf{u}}} - \mathbf{K}\tilde{\mathbf{u}}$ con $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma\Delta t\mathbf{C} + \beta\Delta t^2\mathbf{K}$. Factorización *única* al inicio del análisis y reusada en todos los pasos (**FactorizedSolver**, ADR 0003 fase 2).
- Corrector: $\mathbf{u}_{n+1} = \tilde{\mathbf{u}} + \beta\Delta t^2\ddot{\mathbf{u}}_{n+1}$, $\dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}} + \gamma\Delta t\ddot{\mathbf{u}}_{n+1}$.

Amortiguamiento Rayleigh $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$, declarado de dos formas:

- Directa: `rayleigh: {alpha: 0.1, beta: 1.0e-4}`.
- Calibración modal: `rayleigh: {xi1: 0.02, omega1: 50.0, xi2: 0.02, omega2: 200.0}` — Fenix resuelve los (α, β) que reproducen los amortiguamientos ξ_1, ξ_2 a las frecuencias ω_1, ω_2 .

Parámetros: `t_end`, `dt` (obligatorios); `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (opcional), `u0`, `u0_dot` (condiciones iniciales, default ceros), `F_func` (callback Python $t \rightarrow \mathbf{F}(t)$; default vibración libre).

Salida: **TransientResult** con `t_history`, `u_history`, `udot_history`, `uddot_history` (forma $(n_{\text{dof}}, n_{\text{steps}} + 1)$) y los $\alpha_{\text{Rayleigh}}, \beta_{\text{Rayleigh}}$ efectivos.

```
solver:
  type: NewmarkSolver
  t_end: 2.0
  dt: 0.005
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 50.0
    xi2: 0.02
    omega2: 200.0
```

Limitaciones: lineal ($\mathbf{K}, \mathbf{M}$ constantes; para no-linealidad material o geométrica $\rightarrow$ **NewtonNewmarkSolver**). Apoyos Dirichlet constantes en el tiempo. Paso Δt fijo (adaptativo diferido). HHT- α y generalized- α (con amortiguamiento numérico controlado) no incluidos.

7.3 NewtonNewmarkSolver — transitorio no lineal Newton-Newmark

Subclase de `NewmarkSolver` (Reglas §4 — variante de componente existente) que añade un bucle Newton-Raphson dentro de cada paso temporal sobre el residuo dinámico:

$$\mathbf{R} = \mathbf{F}_{\text{ext}}(t) - \mathbf{F}_{\text{int}}(\mathbf{u}) - \mathbf{C}\dot{\mathbf{u}} - \mathbf{M}\ddot{\mathbf{u}}, \quad \mathbf{J} = \mathbf{M} + \gamma\Delta t \mathbf{C} + \beta\Delta t^2 \mathbf{K}_t$$

Convergencia dual (ADR 0007): mismas tolerancias `rtol_force` / `rtol_disp` y factores `atol*_factor` que los solvers no lineales estáticos. Tras converger cada paso se invoca `assembler.commit_all_stat` (trial → committed en todos los elementos); si agota `max_iter` se lanza `RuntimeError` y se descarta el estado trial.

Amortiguamiento Rayleigh constante en el tiempo, calibrado con la rigidez elástica de referencia $\mathbf{K}_0$ al inicio del análisis. Convención estándar (Abaqus, ANSYS, OpenSees) que evita acoplamiento ad-hoc entre disipación viscosa y plástica.

Newton modificado opcional (`freeze_tangent_after_iter`, ADR 0003 fase 2): factoriza fresco las primeras N iter y reusa la factorización en las siguientes para abaratar pasos largos.

Recuperación del caso lineal: con materiales sin historia o no plastificados, el residuo se anula en 1 iter y el resultado coincide *a paridad de bits* con `NewmarkSolver`. Validado en tests.

Parámetros: heredados de `NewmarkSolver` más `max_iter` (default 20), bloque `convergence` (`rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`; ADR 0007) y `freeze_tangent_after_iter` (default None). El despacho YAML es automático: como `NewtonNewmarkSolver` es subclase de `NewmarkSolver`, `fenix.run_yaml` lo detecta vía `isinstance` y enruta a `run_transient`.

```
materials:
- {id: 1, type: VonMises2D, E: 200.0e9, nu: 0.3, sigma_y: 250.0e6, H: 2.0e9,
  hypothesis: plane_strain, density: 7850.0}

solver:
  type: NewtonNewmarkSolver
  t_end: 0.5
  dt: 0.001
  beta: 0.25
  gamma: 0.5
  rayleigh:
    xi1: 0.02
    omega1: 100.0
    xi2: 0.02
    omega2: 500.0
  max_iter: 15
  convergence:
    rtol_force: 1.0e-6
    rtol_disp: 1.0e-6
```

Cuándo usarlo: respuesta dinámica de estructuras con plasticidad transitoria (sísmica con disipación plástica, impacto en hormigón friccional, fatiga de bajo ciclo), vibraciones de marcos elastoplásticos. Materiales con historia disponibles en dinámica: `Elastoplastic1D`, `VonMises2D` (*plane strain* y *plane stress*), `DruckerPrager2D`, `IsotropicDamage1D/2D`.

Limitaciones: igual que `NewmarkSolver` en lo lineal (apoyos constantes, paso fijo, sin HHT- α). Además: no resuelve *snap-back* dinámico con softening severo — combinar con `ArcLengthSolver` si emerge.

Capítulo 8

Post-Procesamiento y Visualización

8.1 Formato VTU (VTK XML)

Mediante `meshio`, Fenix FEM exporta los resultados en archivos `.vtu` compatibles con ParaView. La información se clasifica en dos diccionarios estándar:

8.1.1 Datos Nodales (Point Data)

- **Displacements:** campo vectorial (u_x, u_y, u_z) por nodo.
- **External_Forces:** distribución nodal del vector de cargas aplicado en el paso actual.
- **Supports:** indicador booleano de DOFs restringidos.

8.1.2 Datos de Elemento (Cell Data)

- **Von_Mises:** tensión equivalente J2 (cuando aplica).
- **Internal_State:** variable principal del material según su `PRIMARY_STATE_VAR` (acumulada equivalente α en plasticidad, d en daño, etc.).
- **Sigma_XX, Sigma_YY, Tau_XY:** componentes del tensor de esfuerzos en notación Voigt.

8.2 Animación con Archivo PVD

Cuando se exporta con `frequency: "all_steps"`, además de un `.vtu` por paso se genera un archivo maestro `.pvd` (XML colección) que ParaView abre como animación temporal. El "tiempo" en el archivo PVD coincide con el factor de carga λ .

8.3 Salida en Texto Plano (Estilo FEAP)

Opcional para verificación manual o post-procesamiento con scripts. Configurable por nodos y elementos seleccionados:

```
output:
  text_export:
    export: true
    nodes: [1, 5, 10] # o "all"
```



```
elements: all
nodal_results: ["Displacements", "External_Forces"]
element_results: ["Von_Mises", "Internal_State"]
```

8.4 Workflow ParaView

1. File >Open y seleccionar el archivo .pvd (o el grupo de .vtu).
2. *Apply* en el panel *Properties*.
3. Reproducir la animación con los controles temporales de la barra superior.
4. Aplicar filtro **Warp By Vector** con **Displacements** para visualizar la deformación física (ajustar **Scale Factor** para amplificar).
5. Cambiar el coloreado de **Solid Color** a la variable de interés (**Von_Mises**, **Internal_State**, **Sigma_XX**, ...).

Capítulo 9

Ejemplos Completos

9.1 Marco 2D — Viga en Voladizo

Combinación de un tramo Euler-Bernoulli y un tramo Timoshenko, empotrada en el extremo y cargada en la punta.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [2.0, 0.0]}
- {id: 3, coords: [4.0, 0.0]}

materials:
- id: 1
  type: Elastic1D
  E: 200.0e9
  density: 7850.0

elements:
- id: 1
  type: Frame2DEuler
  material: 1
  nodes: [1, 2]
  A: 0.01
  I: 8.33e-6

- id: 2
  type: Frame2DTimoshenko
  material: 1
  nodes: [2, 3]
  A: 0.01
  I: 8.33e-6
  As: 0.00833

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0, rz: 0.0}

point_loads_by_node:
- {node_id: 3, uy: -15000.0}

solver:
  type: LinearSolver

output:
```

```

file_name: "viga_voladizo_marco"
pre_process: {export: true}
results:
  frequency: "last_step"
  nodal_results: ["Displacements"]

```

9.2 Placa Continua con Plasticidad J2 (Gmsh)

Placa con perforación central importada desde `.msh`, sometida a carga axial creciente hasta superar el límite elástico del acero. Solver no lineal con paso adaptativo.

```

mesh: "placa_agujero.msh"

mesh_physical_groups:
  "Superficie_Placa":
    material: 1
    thickness: 0.05
    quadrature: "2x2"

materials:
  - id: 1
    type: VonMises2D
    E: 200.0e9
    nu: 0.3
    sigma_y: 250.0e6
    H: 1.5e9
    hypothesis: plane_strain
    density: 7850.0

boundary_conditions_by_group:
  "Borde_Fijo": {ux: 0.0, uy: 0.0}

point_loads_by_group:
  "Borde_Carga": {ux: 1500000.0}

solver:
  type: NonlinearSolver
  num_steps: 30
  max_iter: 10
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

output:
  file_name: "estudio_placa"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
    element_results: ["Von_Mises", "Internal_State"]

```

Interpretación: durante los primeros pasos (régimen elástico) Newton-Raphson converge en 1–2 iteraciones. Cuando la concentración de esfuerzos en la periferia del orificio supera $\sigma_y = 250$ MPa, comienzan iteraciones del *return mapping*; `Internal_State` (la deformación plástica acumulada α) crece visiblemente alrededor del agujero.

9.3 Cable Pretensado en Régimen Corotacional

Cable 2D con material unilateral, cargado transversalmente en su nodo central. Demuestra el acoplamiento elemento corotacional + material unilateral.

```
nodes:
- {id: 1, coords: [0.0, 0.0]}
- {id: 2, coords: [5.0, 0.0]}
- {id: 3, coords: [10.0, 0.0]}

materials:
- id: 1
  type: CableMaterial1D
  E: 150.0e9
  density: 7850.0

elements:
- id: 1
  type: Cable2DCorot
  material: 1
  nodes: [1, 2]
  A: 5.0e-5

- id: 2
  type: Cable2DCorot
  material: 1
  nodes: [2, 3]
  A: 5.0e-5

boundary_conditions_by_node:
- {node_id: 1, ux: 0.0, uy: 0.0}
- {node_id: 3, ux: 0.0, uy: 0.0}

point_loads_by_node:
- {node_id: 2, uy: -500.0}

solver:
  type: NonlinearSolver
  num_steps: 50
  max_iter: 25
  adaptive: true
  convergence:
    rtol_force: 1.0e-5
    rtol_disp: 1.0e-5

output:
  file_name: "cable_carga_transversal"
  pre_process: {export: true}
  results:
    frequency: "all_steps"
    nodal_results: ["Displacements"]
```

Capítulo 10

Uso Avanzado: API Programática

Aunque el flujo principal es *data-driven* (YAML), el motor se puede usar como librería Python para escenarios donde el archivo no es suficiente: optimización paramétrica, generación procedural de mallas, acoplamiento con bibliotecas externas, scripting de batería de tests.

10.1 Patrón Fachada

Los componentes principales se exponen desde la raíz del paquete:

```
import numpy as np
from fenix import Domain, VonMises2D, Quad4, Assembler, NonlinearSolver, VtkExporter

# 1. Dominio y nodos
domain = Domain()
n1 = domain.add_node(1, [0.0, 0.0])
n2 = domain.add_node(2, [1.0, 0.0])
n3 = domain.add_node(3, [1.0, 1.0])
n4 = domain.add_node(4, [0.0, 1.0])

# 2. Material y elemento
acero = VonMises2D(E=200e9, nu=0.3, sigma_y=250e6, H=2e9, hypothesis='plane_strain')
placa = Quad4(element_id=1, nodes=[n1, n2, n3, n4], material=acero, thickness=0.1)
domain.add_element(placa)

# 3. Restricciones (Dirichlet)
n1.fix_dof('ux', 0.0); n1.fix_dof('uy', 0.0)
n4.fix_dof('ux', 0.0); n4.fix_dof('uy', 0.0)

domain.generate_equation_numbers()

# 4. Cargas (Neumann)
F_ext = np.zeros(domain.total_dofs)
F_ext[n2.dofs['ux']] = 150000.0
F_ext[n3.dofs['ux']] = 150000.0

# 5. Resolución
assembler = Assembler(domain)
solver = NonlinearSolver(assembler, num_steps=10, adaptive=True)
U = solver.solve(F_ext)

# 6. Exportación
VtkExporter(domain).export("resultados.vtu", U=U, F_ext=F_ext)
```

Capítulo 11

Diagnóstico de Problemas

- **YamlValidationError: parámetro 'X' no aceptado por 'Y'.**

El parser detecta un parámetro que el constructor del elemento no admite. La salida lista los parámetros aceptados; revisar el catálogo o la spec del componente.

- **Matriz singular detectada.**

Restricciones cinemáticas insuficientes: el dominio puede trasladarse o rotar como cuerpo rígido. Verificar que al menos un nodo tiene fijos los suficientes DOFs para anclar el dominio. En modelos con cables o materiales con softening pronunciado, comprobar que existen caminos de carga alternativos al elemento que se destensa o degrada.

- **Newton-Raphson no convergió / bisecando incremento.**

El paso de carga es demasiado grande. Aumentar `num_steps`, asegurar `adaptive: true`. Verificar tipeo de los parámetros constitutivos (σ_y , κ_0 , etc.). Si el problema tiene snap-through o softening, cambiar a `ArcLengthSolver`.

- **No se importó ningún elemento (Solid2D).**

El parser de Gmsh no encontró superficies bidimensionales. En Gmsh, crear una *Plane Surface* y generar la malla 2D antes de exportar.

- **Cable totalmente destensado --- $KT = 0$.**

Pretensar el cable o garantizar que la estructura tiene rigidez residual por otros elementos. Considerar también pasos de carga finos para capturar la transición tensado $\rightarrow$ destensado.

Fenix FEM — Manual de Usuario

Sigla del manual: FF-MU

Compilado el 13 de mayo de 2026, 16:10

Commit Git: 54e69fe (con cambios sin confirmar)

*Documento generado automáticamente desde los archivos fuente en
manuals/sources/user/ mediante `python manuals/build_user_manual.py`.
No editar directamente el PDF: cualquier corrección debe realizarse sobre el
archivo fuente correspondiente y regenerar el manual.*